웹 기술 이론 5장

JavaScript 언어의 기초 이론

값과 타입, 형변환의 규칙, 스코프와 클로저, 함수라는 값

한신대학교 AI·SW계열 · 웹프로그래밍
학습사이트 hs-web.dreamitbiz.com

학습 목표

① 기본형과 참조형의 저장·복사 방식 차이를 설명할 수 있다.

② 암묵적 형변환이 일어나는 규칙을 알고 함정을 피할 수 있다.

③ 스코프와 호이스팅이 무엇인지 설명할 수 있다.

④ 함수가 값이라는 것과 클로저의 쓰임을 설명할 수 있다.

⑤ 동등 비교(==)를 피해야 하는 이유를 규칙으로 설명할 수 있다.

이론 5장

핵심 키워드

기본형 · 참조형 · 형변환 · truthy/falsy · 스코프 · 호이스팅 · TDZ · 클로저 · 일급 함수 · this

학습사이트의 본문에서 이 낱말들을 찾아 읽으면 이 시간의 뼈대가 잡힙니다

이론 5장 · 1

값과 타입

JavaScript는 동적 타입 언어입니다.

값과 타입

1. 변수에 타입을 미리 정하지 않고, 값이 타입을 갖습니다.
2. 편리하지만 "어떤 타입인지 모른 채 연산하는" 실수가 쉽게 생깁니다.

1.1 기본형과 참조형

	기본형 (primitive)	참조형 (reference)
종류	string · number · boolean · null · undefine…	object · array · function
저장	값 자체	값이 있는 곳의 주소
복사	값이 복사됨 — 서로 독립	주소가 복사됨 — 같은 것을 가리킴
비교	값이 같으면 === 참	같은 것이어야 참
변경	불변(immutable)	가변(mutable)

가장 헷갈리는 지점

```
// 기본형
let a = 1;
let b = a;
b = 2;
console.log(a);      // 1 — 그대로

// 참조형
const x = { n: 1 };
const y = x;
y.n = 2;
console.log(x.n);    // 2 — 같이 바뀐다

// 비교도 다르다
console.log([1,2] === [1,2]);   // false — 내용이 같아도 다른 객체
const arr = [1,2];
console.log(arr === arr);       // true
```

복사하는 법

```
const shallow = { ...x };            // 얕은 복사 — 한 겹만
const arrCopy = [...arr];
const deep = structuredClone(x);     // 깊은 복사 — 안쪽 객체까지

/* 얕은 복사는 안에 또 객체가 있으면 그건 여전히 공유합니다.
   const a = { p: { n: 1 } };
   const b = { ...a };
   b.p.n = 2;  →  a.p.n 도 2가 됩니다 */
```

1.1 기본형과 참조형

1. const 가 막는 것은 재대입이지 내용 변경이 아닙니다.
2. const arr = [1]; arr.push(2) 는 됩니다.
3. arr = [2] 만 오류입니다.

이론 5장 · 2

형변환

직접 바꾸기

```
Number('42')          // 42
Number('42px')        // NaN   ← 통째로 못 바꾸면 NaN
parseInt('42px')      // 42    ← 앞에서부터 숫자만
parseFloat('3.14m') // 3.14
Number('')            // 0     ← 빈 문자열은 0! (주의)

String(42)            // '42'
(42).toString()       // '42'
42 + ''               // '42'

Boolean(0)            // false
!!'hello'             // true  ← 두 번 부정으로 불리언 변환
```

+ 연산자의 두 얼굴

```
'5' + 3      // '53'   ← 한쪽이 문자열이면 이어붙이기
'5' - 3      // 2      ← -, *, / 는 숫자로 변환
'5' * '2'    // 10
'a' * 2      // NaN

1 + '2' + 3  // '123'  ← 왼쪽부터 순서대로
1 + 2 + '3'  // '33'   ← 앞의 1+2가 먼저 계산됨

[] + []      // ''
[] + {}      // '[object Object]'
true + true  // 2      ← 불리언도 숫자로
```

2.2 암묵적 변환 — 함정의 근원

1. 이 규칙을 다 외울 필요는 없습니다.

2. 실무 대응은 하나뿐입니다 — 연산 전에 타입을 맞춥니다.

실무의 방어

```
// input.value 는 type="number" 여도 항상 문자열입니다
const price = Number(document.querySelector('#price').value);
const qty = Number(document.querySelector('#qty').value);

if (Number.isNaN(price)) { /* 숫자가 아닌 입력 처리 */ }
const total = price * qty;   // 이제 안전
```

falsy는 여섯 개뿐입니다

```
false, 0, '', null, undefined, NaN

// 나머지는 전부 truthy — 특히 이것들이 참입니다
if ([])        console.log('빈 배열도 참');    // 실행됨
if ({})        console.log('빈 객체도 참');    // 실행됨
if ('0')       console.log('문자열 0도 참');   // 실행됨
if (-1)        console.log('음수도 참');       // 실행됨
```

기본값 주기 — || 와 ?? 의 차이

```
const count1 = value || 10;   // value가 falsy면 10 → 0이나 ''도 10이 됨
const count2 = value ?? 10;   // value가 null · undefined일 때만 10 → 0은 0 그대로

/* 숫자 · 문자열을 다룰 때는 ?? 를 쓰는 편이 안전합니다.
   "재고 0개"가 "재고 10개"로 바뀌는 버그가 실제로 자주 납니다. */
```

동등 비교의 변환 규칙

```
'1' == 1            // true
0 == false          // true
'' == false         // true
null == undefined   // true
null == 0           // false  ← 일관성도 없다
NaN == NaN          // false  ← 자기 자신과도 다르다

/* 규칙이 복잡하고 예외가 많습니다.
   === 를 쓰면 이 모든 것이 false — 예측 가능해집니다. */
```

2.4 == 를 쓰지 않는 이유

1. 유일한 예외로 x == null 은 null과 undefined를 한 번에 검사하는 관용 표현으로 쓰기도 합니다.
2. 그 외에는 항상 === 입니다.

이론 5장 · 3

스코프 — 변수가 사는 범위

스코프 — 변수가 사는 범위

종류	범위	누가
전역 스코프	문서 전체	최상위 선언
함수 스코프	그 함수 안	var · 함수 매개변수
블록 스코프	중괄호 {} 안	const · let

var가 문제인 이유

```
if (true) {
  var a = 1;
  const b = 2;
}
console.log(a);   // 1  ← var는 블록을 새어 나온다
console.log(b);   // ReferenceError  ← 정상

// 반복문에서 특히 심각합니다
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// 3, 3, 3 ← i가 하나뿐이라 반복이 끝난 값이 찍힌다

for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// 0, 1, 2 ← let은 반복마다 새 i를 만든다
```

선언이 위로 끌어올려지는 현상

```
console.log(a);   // undefined  ← 선언은 올라가고 값은 안 올라감
var a = 1;

console.log(b);   // ReferenceError: Cannot access 'b' before initialization
let b = 2;
/* const/let도 호이스팅되지만 선언 전까지는 접근이 막혀 있습니다.
   이 구간을 TDZ(Temporal Dead Zone)라고 합니다.
   var처럼 조용히 undefined가 나오지 않고 오류를 내 주므로 오히려 안전합니다. */

foo();            // 정상 실행 — 함수 선언은 통째로 올라간다
function foo() {}

bar();            // TypeError — 값이 아직 없다
const bar = () => {};
```

이론 5장 · 4

함수는 값이다

JavaScript에서 함수는 일급(first-class) 값입니다.

함수는 값이다

1. 변수에 담고, 인자로 넘기고, 반환할 수 있습니다.

2. map·filter·addEventListener 가 함수를 인자로 받는 것이 모두 이 성질 덕분입니다.

함수를 값처럼 다루기

```
const greet = (name) => `안녕, ${name}`;   // 변수에 담기

[1, 2, 3].map(n => n * 2);                 // 인자로 넘기기

function makeAdder(x) {                    // 반환하기
  return (y) => x + y;
}
const add5 = makeAdder(5);
add5(3);   // 8
```

4.1 클로저

1. 위 makeAdder 에서, 돌려받은 함수는 자기가 만들어진 곳의 변수 x 를 기억하고 있습니다.
2. 이것을 클로저(closure) 라고 합니다.

클로저의 실전 쓰임 — 상태 감추기

```
function makeCounter() {
  let count = 0;                    // 밖에서 직접 못 건드린다
  return {
    up:    () => ++count,
    down:  () => --count,
    value: () => count,
  };
}

const c = makeCounter();
c.up(); c.up();
console.log(c.value());   // 2
console.log(c.count);     // undefined — 밖에서는 접근 불가
```

4.1 클로저

1. 이벤트 핸들러 안에서 바깥 변수를 쓸 수 있는 것도 클로저 덕분입니다.

2. 11주차에 만든 삭제 버튼이 자기 인덱스를 기억하는 것이 그 예입니다.

결정적인 차이

```
const obj = {
  name: '한신',

  a: function () { console.log(this.name); },   // '한신' — 호출한 객체
  b: () => { console.log(this.name); },         // undefined — 바깥 this를 그대로 씀
};

/* 화살표 함수는 자기 this를 만들지 않습니다.
   → 객체의 메서드에는 일반 함수, 콜백에는 화살표 함수가 대체로 맞습니다. */

// 콜백에서는 화살표가 편합니다
class Timer {
  constructor() { this.n = 0; }
  start() {
    setInterval(() => { this.n++; }, 1000);   // this가 Timer 인스턴스 그대로
  }
}
```

4.2 화살표 함수와 this

1. this 는 JavaScript에서 가장 헷갈리는 주제이지만, 이 과목 범위에서는 "화살표 함수는 자기 this가 없다" 만 알면 충분합니다.

2. 우리가 쓰는 콜백에서는 그것이 오히려 편리하게 동작합니다.

이론 5장 · 5

실무에서 자주 쓰는 최신 문법

알아 두면 코드가 짧아지는 것들 (1/2)

```
// 구조 분해
const { name, year = 1 } = student;
const [first, ...rest] = [1, 2, 3];

// 펼침
const merged = { ...base, year: 2 };
Math.max(...[3, 1, 4]);

// 선택적 체이닝 — 없어도 오류가 안 남
const city = user?.address?.city;
obj.method?.();
```

알아 두면 코드가 짧아지는 것들 (2/2)

```
// 널 병합
const n = value ?? 0;

// 축약 속성
const name2 = '한신';
const obj2 = { name2 };            // { name2: '한신' } 과 같음

// 계산된 속성 이름
const key = 'year';
const obj3 = { [key]: 1 };         // { year: 1 }
```

웹 기술 이론 5장 마무리

5장 마무리

JavaScript 언어의 기초 이론 — 본문과 도해는 학습사이트에서 다시 읽습니다

- ✓ 학습사이트 장 페이지 끝의 「이해했어요 · 어려워요」 표시
- ✓ 함께 볼 주차: 9주차 · 10주차
- ✓ 막히면 「질문있어요」 — 질문은 반 전체가 같은 자리에서 봅니다